

Web-Security

Authentifikation und Autorisierung

HTTP Tokens

Token-basierte Authentifikation (Authorization)

Moderne Web-Schnittstellen verwenden gerne eine besondere Art der Autorisierung, die zwingend mit HTTPS verwendet werden sollte

- Basis ist ein Token, eine Art Geheimnis, welches man mit der Anfrage mit präsentieren muss
- Dieses Geheimnis erhält man nach erfolgreicher Authentifikation, ggf. auch bei einem externen Anbieter und speichert dieses zwischen
- Token hat eine Struktur und eine Signatur
- Token wird als **HTTP-Authorization-Header** vom Typ **Bearer** übertragen

JSON Web Token (eine Möglichkeit für Tokens)

- aaaaaaaaaa.bbbbbbbbbbbb.cccccccccccc (jeweils base64-codiert)
- Header.payload.signature
- Header gibt in JSON-Format an: JWT und Hash-Algorithmus
- Payload liefert wichtige Information (Claims): Subject, Issuer, Audience, Expiration, Not-valid-before, Ausstellzeitpunkt
- Signatur über Header, Payload und zusätzlichem Secret

- Verteiltes **Autorisierungsprotokoll** mit standardisierter Programmierschnittstelle
- Offengelegtes Protokoll
- **Token-basierter Schutz auf Ressourcen**
- 4 Rollen:
 - Resource Owner
 - Resource Server
 - Client
 - Authorization Server
- Zusammen mit JSON Web Tokens (JWT) die de facto Standardlösung zur Autorisierung
- Liefert jedoch eigentlich keine Aussagen über die vorher notwendige Authentifikation!

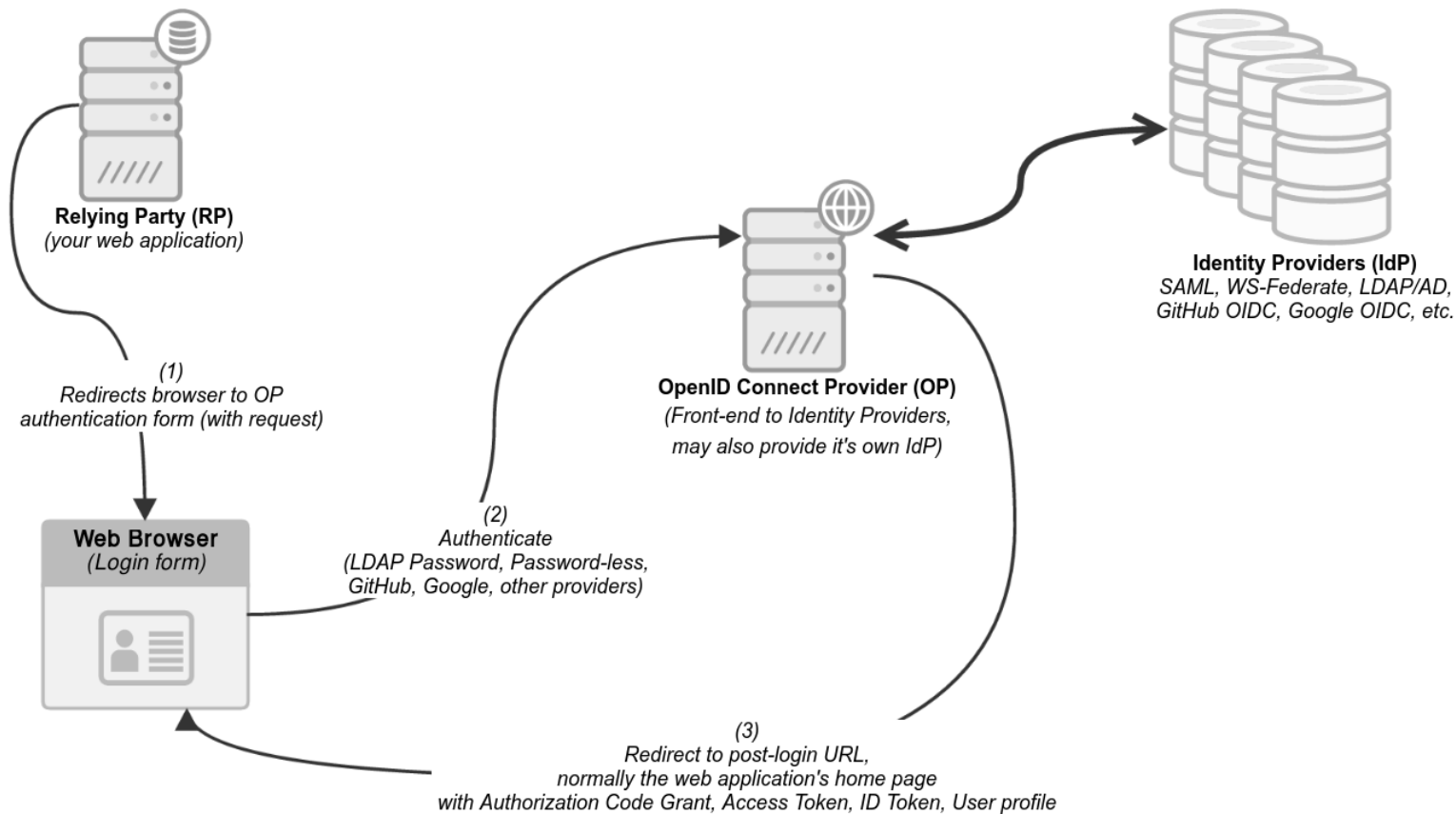
- Verteiltes Authentifizierungssystem in der Web-Landschaft
- Offengelegtes Protokoll, das auf dem OAuth 2.0-Protokoll basiert (Autorisierung einen IdP zu fragen) und dieses im Sinne der Authentifikation erweitert
- Ausschließlich unter OpenSource bereitgestellte Software
- Dient der föderierten Authentifikation
- Relativ einfach für die Nutzer und daher gerne genutzt im Web-Kontext

OpenID Connect nutzt OAuth 2 als Grundlage für die Autorisierung und erweitert dieses um Authentifizierung. Es ermöglicht die Wiederverwendung bestehender Identitäten so dass keine eigene Benutzerdatenbank erforderlich ist

HTTP Tokens

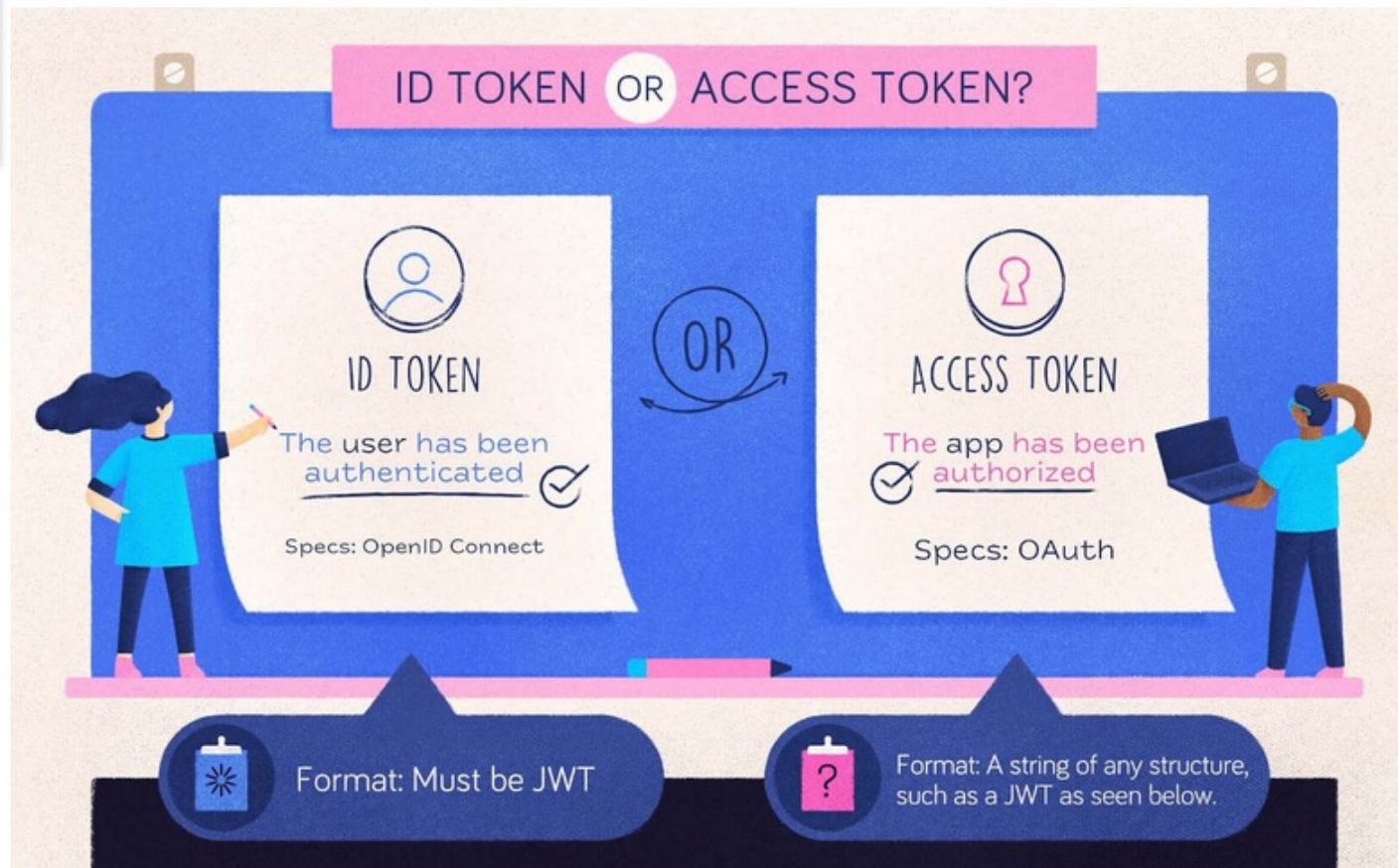
OpenID Connect

https://infosec.mozilla.org/guidelines/iam/openid_connect.html



HTTP Tokens

<https://auth0.com/blog/id-token-access-token-what-is-the-difference/>



HTTP To

<https://auth0.com/bl>



Viele **Identity Provider (Google, Microsoft, Auth0)** geben **nur einen Access Token zurück**, der **beide Funktionen übernimmt**:

- **Authentifizierung**: Enthält `sub`, `email`, `name` (wie ein ID Token)
- **Autorisierung**: Enthält `scopes` (wie ein Access Token)

Viele Entwickler nutzen damit nur **OAuth2 mit implizitem OIDC**, ohne wirklich einen **ID Token** zu validieren

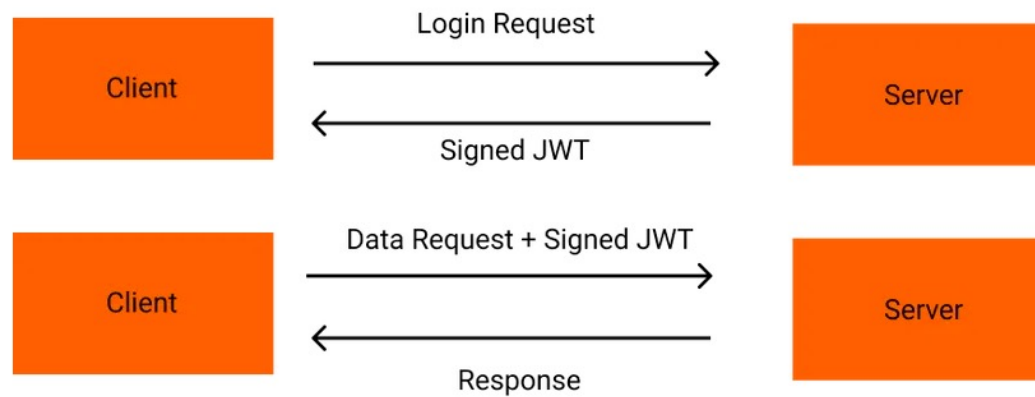
...

HTTP Tokens

JSON Web Token

▶ BODY ⓘ pretty ▾

```
{  
  accessToken: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InZvbGtldiIsInJvbGUiOi0  
}
```



<https://stackabuse.com/authentication-and-authorization-with-jwts-in-express-js/>

GET /books HTTP/1.1

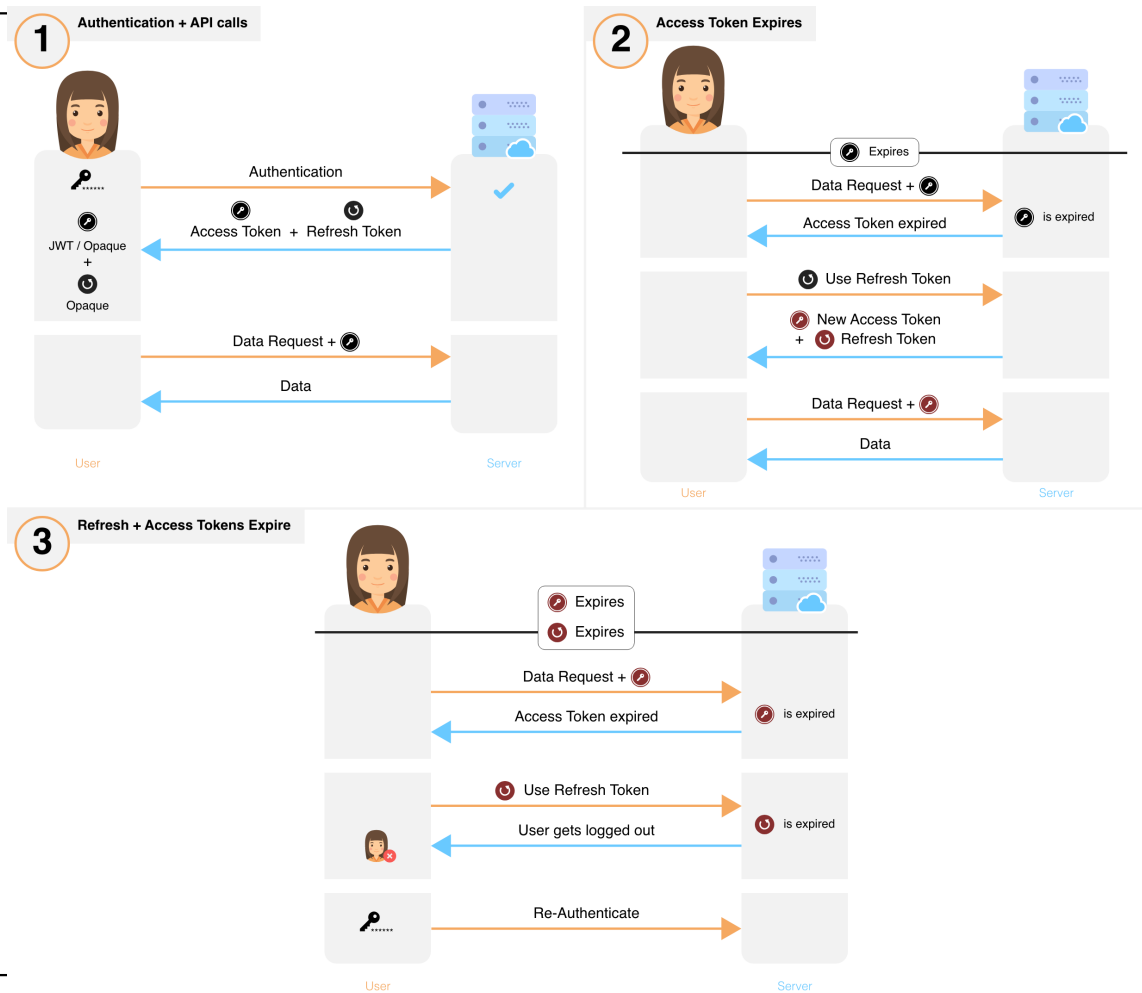
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InZvbGtldiIsInJvbGUiOi0

Host: localhost:3000

HTTP Tokens

Refresh Token

<https://supertokens.com/docs/community/prologue>



HTTP Tokens

JSON Web Token - einfach

```
const express = require('express');
const app = express();
const jwt = require('jsonwebtoken');

const accessTokenSecret = 'mein access token secret';

app.use(express.json());

const users = [
  {
    username: 'volker',
    password: 'password123admin',
    role: 'admin'
  }, {
    username: 'sander',
    password: 'password123member',
    role: 'member'
  }
];
```

HTTP Tokens

JSON Web Token - einfach

```
app.post('/login', (req, res) => {  
  // Read username and password from request body  
  const { username, password } = req.body;  
  console.log("User " + username + " with pw " + password)  
  // Filter user from the users array by username and password  
  const user = users.find( predicate: u => { return u.username === username && u.password === password });  
  
  if (user) {  
    // Generate an access token  
    const accessToken = jwt.sign( payload: { username: user.username, role: user.role }, accessTokenSecret);  
  
    res.json({  
      accessToken  
    });  
  } else {  
    res.send('Username or password incorrect');  
  }  
}
```

HTTP Tokens

JSON Web Token - einfach

```
const authenticateJWT = (req, res, next) => {  
  const authHeader = req.headers.authorization;  
  
  if (authHeader) {  
    const token = authHeader.split(' ')[1];  
  
    jwt.verify(token, accessTokenSecret, options: {err, user}) => {  
      if (err) {  
        return res.sendStatus(403);  
      }  
  
      req.user = user;  
      next();  
    });  
  } else {  
    res.sendStatus(401);  
  }  
};
```

HTTP Tokens

JSON Web Token - einfach

```
const books = [
  {
    "author": "a",
    "language": "aa",
    "title": "Erster"
  },
  {
    "author": "b",
    "language": "bb",
    "title": "Zweiter"
  },
  {
    "author": "c",
    "language": "vc",
    "title": "Dritter"
  }
];

app.get('/books', authenticateJWT, (req, res) => {
  res.json(books);
});

app.listen(3000, () => {
  console.log('Authentication service started on port 3000');
});
```

HTTP Tokens

JSON Web Token

METHOD: POST SCHEME // HOST [":" PORT] [PATH ["?" QUERY]]
http://localhost:3000/login length: 27 byte(s) Send

QUERY PARAMETERS
+ Add query parameter

HEADERS [?] Form
☒ Content-Type : application/json x
+ Add header Add authorization 🗑

BODY [?] Text
1 { "username": "volker", "password": "password123admin" }
Text JSON XML HTML | Format body | ☒ Enable body evaluation 🗑 length: 53 bytes

METHOD: GET SCHEME // HOST [":" PORT] [PATH ["?" QUERY]]
http://localhost:3000/books length: 27 byte(s) Send

QUERY PARAMETERS
+ Add query parameter

HEADERS [?] Form
☒ Authorization : Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6Ii... x 🗑
+ Add header Add authorization

BODY [?]
XHR does not allow payloads for GET request.

HTTP API-Keys

Identifikation von Anwendung

Tokens sind private, personenbezogene Hilfsmittel

API-Keys sollen eine Anwendung oder ein Projekt identifizieren

- Faktisch werden API-Keys gerne dazu verwendet, die Aufrufe Anwendungen, Projekten oder doch gar Personen zuzuordnen. Diese Zuordnung ist jedoch locker und eher für statistische Zwecke geeignet, nicht um den Zugang wirklich bestimmten Benutzern zuzuordnen
- API Keys werden einmalig erzeugt, Tokens kontinuierlich
- Beispiel: Google Maps benötigt ein API-Key, den man sich erzeugen muss (der erzeugt wird)

★ **New Users:** Before you can start using the Google Maps Platform APIs and SDKs, you must sign up and create a billing account. To learn more, see [Get Started with Google Maps Platform](#).

To use the Maps JavaScript API you must have an API key. The API key is a unique identifier that is used to authenticate requests associated with your project for usage and billing purposes.

<https://developers.google.com/maps/documentation/javascript/get-api-key>